

Project Kinetica

A focused Technical Art study connecting weapon motion, impact evaluation, vertex deformation and material-specific particles.

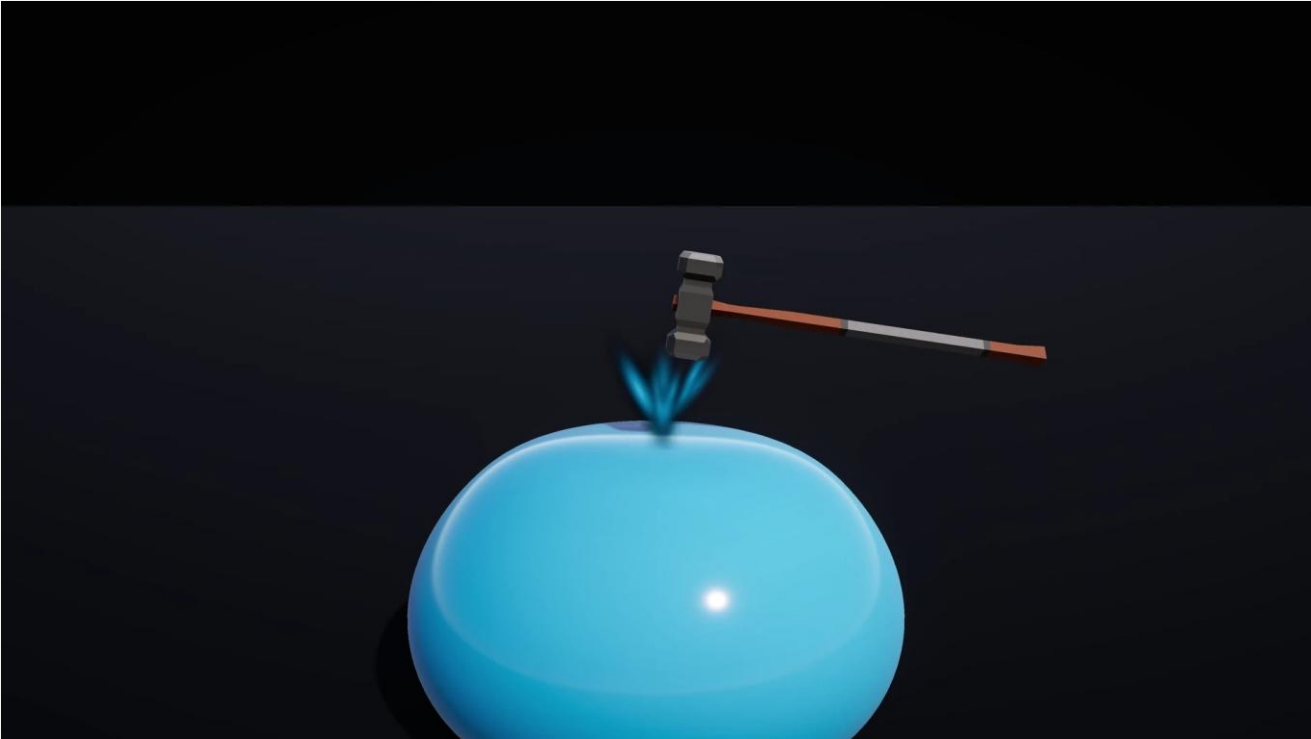


Fig. 01 — Final Hammer–Slime impact response.

ROLE	ENGINE	TOOLS	STATUS
Technical Art · Programming	Unity 6 · URP	C# · Shader Graph · Particle System	Frozen implementation

Kinetica evaluates one authored contact and translates it into a localized surface response plus a short material-specific particle burst. The completed study covers Hammer, Sword and Spear interactions with Slime, Stone and Metal. It is an art-directed response model rather than a calibrated physical simulation.

Implementation source: [Dull-Pigeon/Project-Kinetica · commit eb011e727cf2e4299ad9a5585ee6a5ecf75b9e35](#)

Core System Architecture

Configuration, collision sampling, impact evaluation and presentation have separate responsibilities. Deformation is a direct call from WeaponCollision; particle and debug systems observe the static impact event.

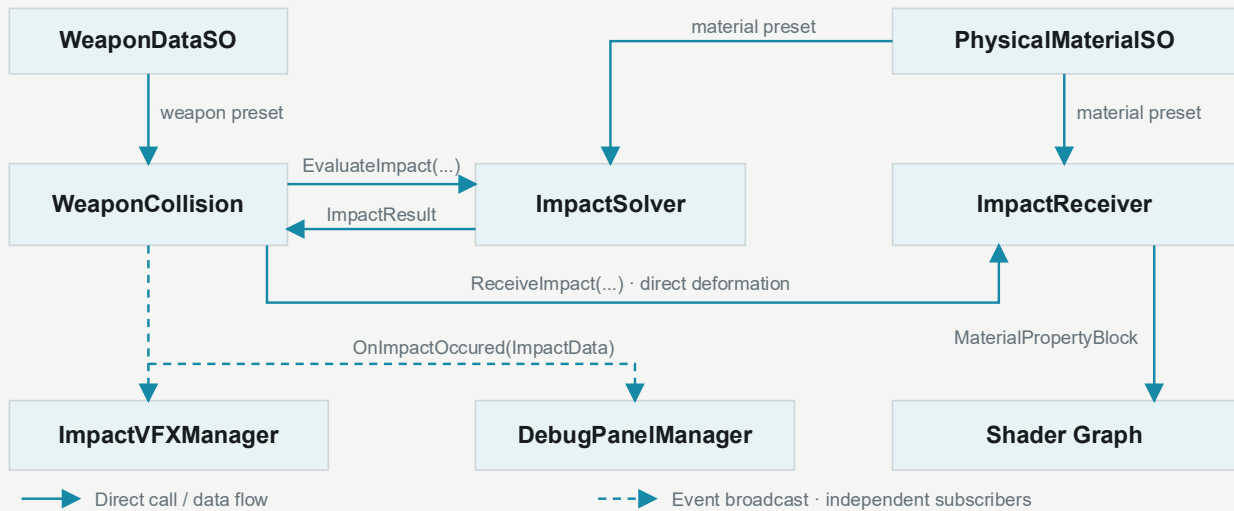


Fig. 02 — Frozen runtime architecture. Solid lines show calls and data flow; dashed branches broadcast to independent VFX and Debug subscribers.

Data responsibilities

WeaponDataSO stores weapon name, mass, hardness, contact area and detection radius. PhysicalMaterialSO stores material name, hardness, elasticity, critical pressure and the impact VFX prefab. These reusable ScriptableObject presets keep authored settings outside the runtime components. Separating weapon and target data allows the same weapon to be evaluated against different materials without duplicating collision or solver logic; a material can also be reused across targets.

Authored parameters

WEAPON	MASS	HARDNESS	AREA	DETECTION RADIUS
Hammer	6	9	0.8	0.2
Sword	4	7	0.3	0.1
Spear	5	8	0.2	0.1

MATERIAL	HARDNESS	ELASTICITY	CRITICAL PRESSURE
Slime	1.5	0.85	30
Stone	7.5	0.01	60
Metal	8.5	0.65	100

Values are authored controls with no invented units. Area influences pressure and is passed directly to deformation radius; detection radius controls SphereCast thickness.

Weapon Motion and Collision Detection

WeaponCollision samples a serialized detection point in FixedUpdate. It derives velocity from point displacement and uses $\text{mass} \times \text{velocity}$ as a momentum-inspired directional strength input. During an animation-event attack window, SphereCast sweeps the point's travelled segment against the Target layer. A HashSet prevents the same collider from responding twice in one swing.

Sample	Sweep	Gate
Detection point displacement is divided by fixed delta time.	One SphereCast hit is returned along the sampled straight segment.	StartAttack and StopAttack define the active query window.

The query returns one RaycastHit rather than every collider along the sweep. ImpactReceiver must be on the hit collider's GameObject: TryGetComponent is called there, with no parent lookup. The HashSet records collider identities, so several colliders on one logical target can each receive a response. Correct receiver placement and Target-layer assignment are therefore part of the authored setup.

StartAttack clears the hit set and resets the previous sample to the current point; StopAttack closes the query window. Position sampling continues while idle. The sweep follows one straight displacement segment, which approximates sampled movement rather than the complete curved path of a rotating blade.

Impact Solver

The solver normalizes direction vectors and evaluates alignment first. Alignment below 0.25 returns Glanced with zero pressure and deformation force. A softer weapon returns Deflected and reduces deformation force to 20%. Otherwise, pressure is effective force divided by area: below the material threshold returns Squashed; at or above it returns Penetrated and sets isBroken.

01 Alignment	02 Hardness	03 Pressure
Direct inward contact approaches 1; tangential contact approaches 0.	Weapon hardness is compared with the target material.	Threshold classification runs only after the first two checks.

Penetrated and isBroken are classifications. The project does not cut, fracture or destroy geometry. The animated weapon is not physically redirected by a Deflected result. One hit is returned per sweep, and deduplication is per collider rather than per logical target.

Impact payload

For a valid receiver, WeaponCollision calls deformation before broadcasting ImpactData. The readonly payload carries weapon and material references, ImpactResult, impact point, normal and sampled velocity. ImpactVFXManager and DebugPanelManager subscribe in OnEnable and unsubscribe in OnDisable. Glancing results can still publish an event and spawn the material VFX.

The assets describe inputs rather than owning a live contact. WeaponCollision supplies sampled motion and contact geometry, ImpactSolver returns the evaluated result, and ImpactReceiver owns the current shader response. Meshes, rendering materials, animations and lighting still contribute to appearance; the numerical presets do not define the complete visual identity.

Localized Shader Response

ImpactReceiver writes per-renderer parameters through MaterialPropertyBlock. Hardness suppresses the incoming deformation force while elasticity scales amplitude. The receiver also derives frequency, decay, hold time and rebound factor. A later hit overwrites the receiver's single stored impact state.

Material response mappings

For the authored elasticity range, the receiver transforms the solver's hitForce with the following mappings. Hardness reduces displacement, while elasticity scales its amplitude and controls the recovery shape. The Debug Panel reports the earlier solver force, so its value is not the final vector sent to Shader Graph.

PARAMETER	FROZEN MAPPING
deformForce	$\text{hitForce} / (1 + \text{hardness}^2) \times (0.08 \times \text{elasticity})$
frequency	$10 + 40 \times (1 - \text{elasticity})$
decay	$1 + 29 \times (1 - \text{elasticity})$
holdTime	$0.1 / (1 + \text{elasticity})$
reboundFactor	$0.5 \times \text{elasticity}$

Frequency multiplies time directly before Cosine. It is an angular-rate style control, not a frequency labelled in Hz; the graph does not insert a conversion from cycles to radians. These mappings are authored response controls rather than measured material constants.

Spatial Falloff

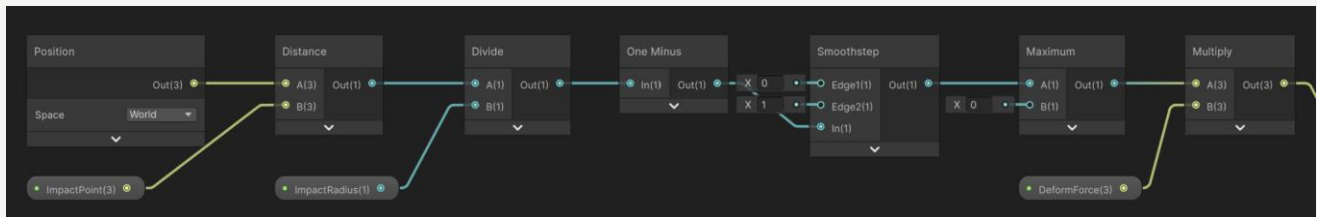


Fig. 03 — Spatial deformation falloff in Shader Graph.

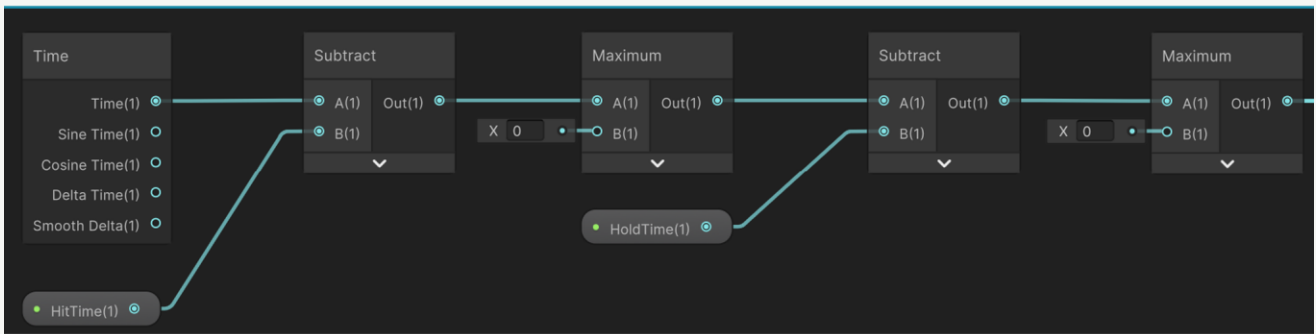
Distance from vertex position to impact point is divided by Impact Radius, subtracted from 1, then shaped with Smoothstep. The frozen C# path passes weapon area directly as the radius.

The spatial weight is $\text{Smoothstep}(0, 1, 1 - \text{distance} / \text{radius})$. It is strongest around the recorded point and falls to zero at the radius boundary. This radial point falloff cannot accurately represent elongated sword-blade contact: increasing the radius broadens a circular region rather than matching the blade's contact shape.

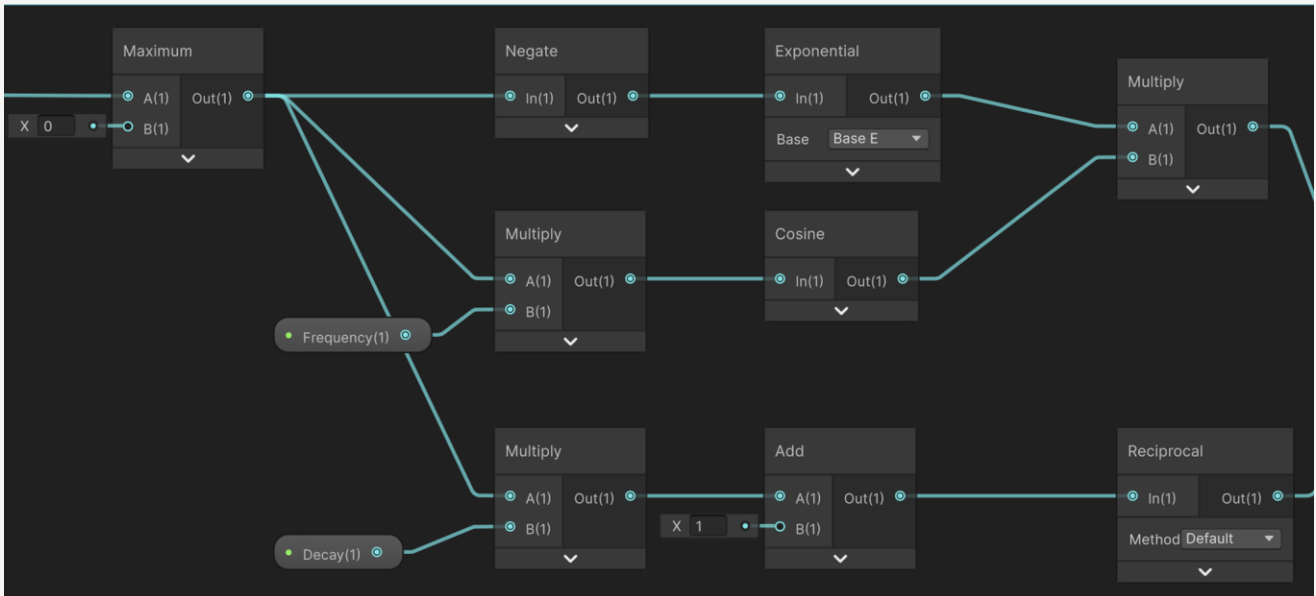
Deformation quality depends on mesh vertex density. The shader moves available vertices; a sparse mesh cannot represent a finely localized indentation between them. Additional subdivision improves spatial sampling but increases vertex work. The graph applies the weighted offset before converting to the vertex output space, while the collider and original topology remain unchanged.

Temporal Deformation Response

A HOLD-TIME OFFSET



B OSCILLATION & DAMPING



C REBOUND SHAPING

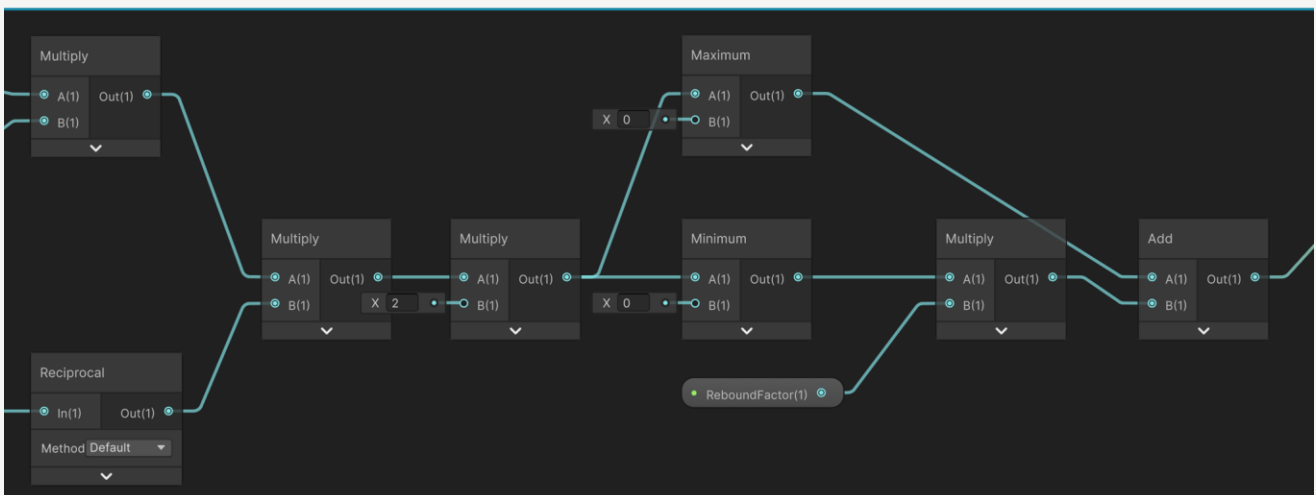


Fig. 04 — Temporal response in Shader Graph: A hold-time offset; B oscillation and damping; C rebound shaping.

The graph clamps elapsed time, subtracts Hold Time and clamps again. The shifted time drives cosine and damping; the connected chain multiplies the oscillation by 2. Positive and negative portions are separated, and only the negative portion is scaled by Rebound Factor before recombination.

Rendered vertices move; mesh topology, collider geometry and displaced normals are unchanged.

Material-specific Visual Response

Each `PhysicalMaterialSO` selects a short, non-looping built-in Particle System prefab. Slime uses stretched billboard droplets with gravity; Stone uses mesh fragments with rotation over lifetime to create tumble; Metal uses stretched billboard spark streaks. The Stone particles are spawned geometry and leave the target intact. These are authored visual languages rather than measurements of material behavior.

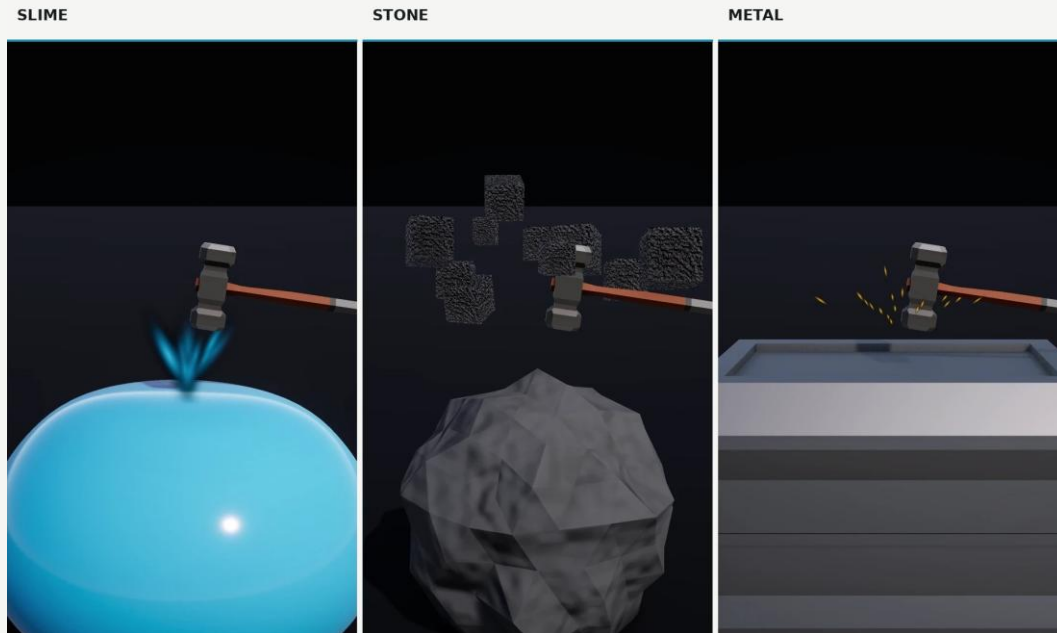


Fig. 05 — Authored material comparison in consistent Slime / Stone / Metal order.

Debug Visualization

`DebugPanelManager` shows the latest event snapshot: authored inputs, alignment, classification, pressure, solver deformation-force magnitude, velocity and impact position. The shown force precedes receiver scaling; Broken repeats a solver flag and does not indicate destroyed geometry.

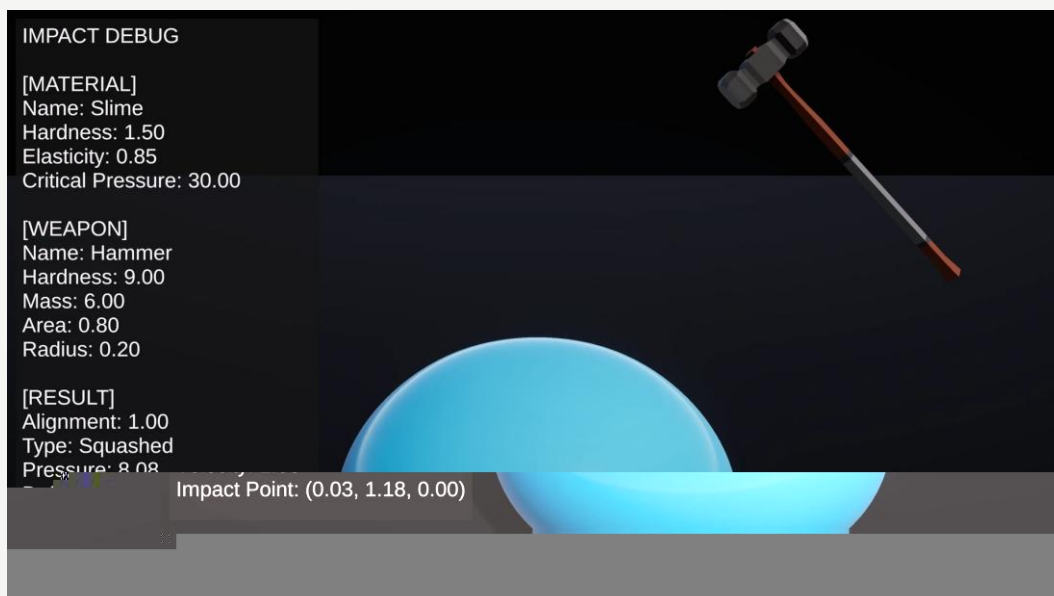


Fig. 06 — Hammer-Slime runtime data for one recorded impact; this panel is not a profiler.

Problems, Iterations and Technical Decisions

Slime topology and shading

Faceted deformation and uneven highlights were addressed through mesh topology and normal revisions. The final footage shows the result, without claiming a controlled before-and-after comparison.

Detection point hierarchy

Imported pivots did not always match the desired contact region. Separating the visible mesh from the sampled detection point kept asset-specific placement outside the solver.

Collision radius versus contact area

A shared early control produced contacts before the broad weapon visually reached the target. Detection radius now tunes the sweep; area controls pressure and remains coupled to visual deformation radius.

Carapace reframed as Metal

Carapace overlapped visually with Stone while its sparks suggested metal. Renaming and retuning the target aligned shape, surface response and particles.

Lighting and effect direction

Removing a competing fill light improved Slime readability. A reflected-velocity VFX direction was replaced by surface-normal orientation because it did not generalize cleanly across contacts.

Performance boundaries

MaterialPropertyBlock provides per-renderer values while retaining the referenced material asset, avoiding a new material instance for each impact. Shader.PropertyToID values are cached. Neither choice establishes zero rendering overhead or guarantees batching; the actual render path still requires measurement.

SphereCast runs only during an attack window with nonzero displacement, although motion sampling continues in FixedUpdate while idle. Query count grows with simultaneously attacking weapons. More mesh vertices can improve deformation detail but increase vertex processing. Transparent particle overlap can increase overdraw, while Stone's mesh particles add geometry cost as well as their visible fragment detail.

ImpactVFXManager uses Instantiate for each event; effects are destroyed after playback and no pooling is implemented. Debug string formatting and UI updates also occur per event. These are cost categories to profile, not measured bottlenecks. The frozen delivery includes no CPU/GPU profiling report or scalability benchmark, so the recorded 60 fps cannot establish runtime performance.

VFX placement and playback

The manager places the selected prefab at the impact point with a fixed 0.1 world-down offset and uses the surface normal for emitter orientation. Prefabs use a short system duration and destroy-on-stop behavior; particle lifetime is configured separately. Neither pressure nor result type changes particle count or selects an alternate effect. A glancing event can therefore still produce particles. The fixed offset is an authored placement adjustment, not a surface-relative correction for every contact orientation.

Results and Showcase

The final 46-second presentation shows individual Hammer interactions, a three-material comparison, a Hammer / Sword / Spear comparison on Slime, and a final diagnostic shot. Captured 4K / 60 fps describes the recording, not measured runtime performance.

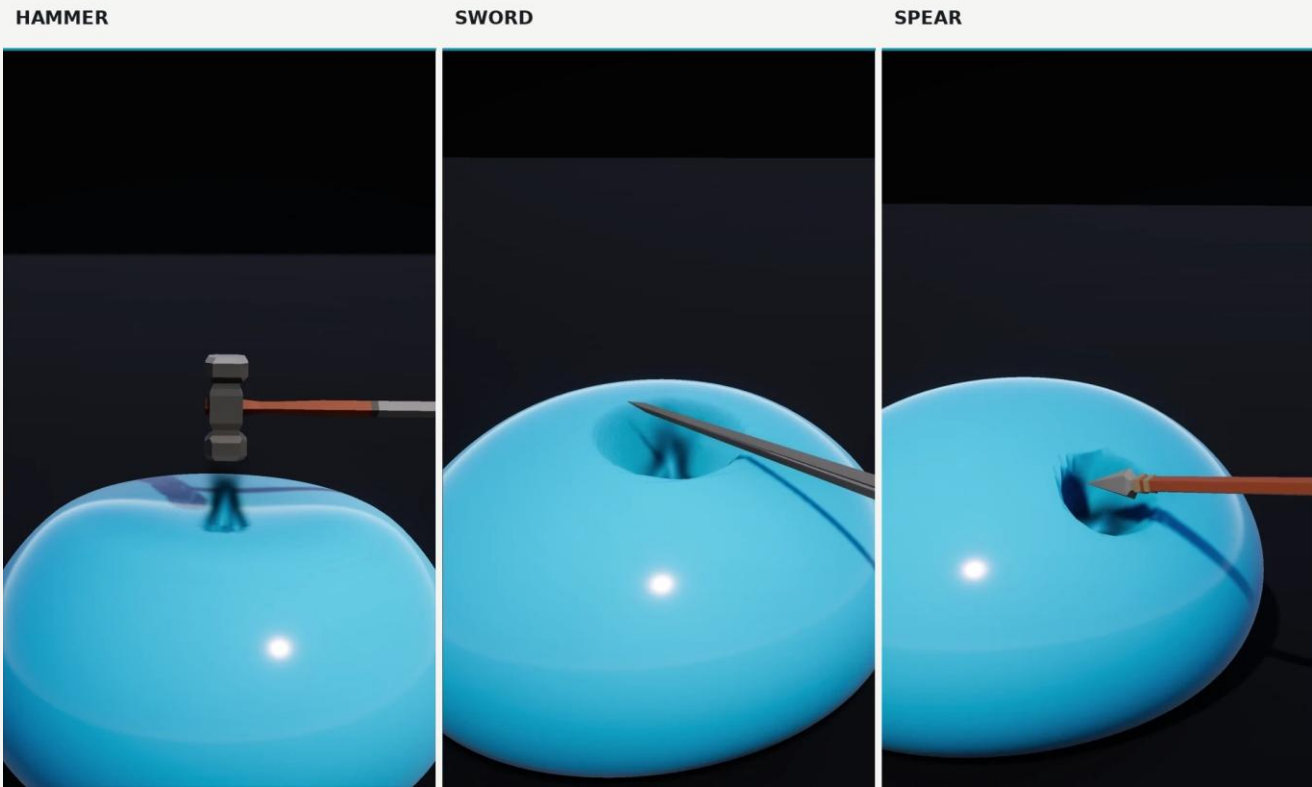


Fig. 07 — Authored weapon comparison, identified as Hammer / Sword / Spear.

Different approach directions and contact locations produce localized responses on the same target. Animation, contact geometry, mass and area also differ, so the montage should not be read as an experiment isolating one physical variable.

Limitations and Scope

Collision	Deformation	Presentation
One swept point, one returned hit per physics step, straight sampled segment and collider-level deduplication.	Rendered positions only; one stored hit; no collider, topology or normal reconstruction.	Material-based particle prefab with no result filter, pressure intensity or pooling.

The model omits calibrated units, impact duration, energy conservation, friction, target mass and structural mechanics. It contains no complete combat loop, damage system, destruction system or performance benchmark. The final evidence supports only the captured authored interactions.

Reflection and Lessons Learned

Readable response depends on the full path from authored motion to presentation. A coherent calculation can still look wrong when the detection point is misplaced, the deformation region is too broad, the mesh cannot support the displacement or lighting hides the contact. Separating the stages made those causes easier to isolate.

The most useful design correction was separating contact area from detection radius because the two values answer different questions. The remaining area-to-deformation-radius coupling is documented as part of the frozen design. Slime reinforced the connection between topology, normals and shader behavior; the Metal revision showed that target form and particles must communicate the same material idea.

The hybrid architecture was sufficient for this study: the receiver stays on a clear direct path while a shared event distributes evaluated data to VFX and debugging. The project also clarified how evidence should be used. Code establishes implementation behavior, footage establishes final appearance, and profiling would establish performance.

Credits and Third-Party Assets

USE	ASSET	CREATOR / SOURCE	LICENSE
Weapon assets	Low Poly Melee & Ranged Medieval Weapons	TUFFPOLYGON · Unity Asset Store	Standard Unity Asset Store EULA
Showcase music	Chill Bro	Diego Nava · Mixkit	Mixkit Free License

Technical implementation, Shader Graph deformation system, system integration, VFX configuration, attack setup, debugging tools, lighting and final presentation: Lin Wenqi, as documented by the project record.

Source record

Frozen implementation: Dull-Pigeon/Project-Kinetica · eb011e727cf2e4299ad9a5585ee6a5ecf75b9e35

Public selected source: github.com/Dull-Pigeon/Project-Kinetica-Public

Visual evidence: final Project Kinetica Showcase. Figures show authored results and one recorded debug event; they are not controlled physical measurements.